ML and DS Internship Practical

Month 4 – Real-World Projects

Submitted By:

Saniya Shaikh

Submitted To: -

Mr. Farhan Memon (Groot Technology)

Technology Used: Python

Development Environment: Visual Studio Code

E-Commerce Backend API

Project Documentation

1. Project Overview

The E-Commerce Backend API is a Python-based backend application developed using Flask. It manages user registration, login, product information, shopping cart operations, and order processing. SQLite is used as the database through Flask-SQLAlchemy.

2. Objectives

- To develop REST APIs for an e-commerce application.
- To manage user registration and login.
- To perform product management operations.
- To manage shopping cart items and quantities.
- To process customer orders and calculate total amounts.
- To store application data in an SQLite database.

3. Technologies Used

- Programming Language: Python
- Framework: Flask
- Database: SQLite
- ORM: Flask-SQLAlchemy
- Password Security: Werkzeug
- API Testing: Thunder Client
- Development Environment: Visual Studio Code

4. Main Features

- User registration and login.
- Add, view, update, and delete products.
- Add products to the shopping cart.

- View, update, and remove cart items.
- Calculate the cart's total amount.
- Place orders and check order history.
- Validate product availability before placing an order.
- Store user passwords as hashes.

5. API Endpoints

Method	Endpoint	Description
POST	/register	Register a new user
POST	/login	Login user
GET	/products	View all products
POST	/products	Add a product
PUT	/products/<product_id>	Update a product
DELETE	/products/<product_id>	Delete a product
POST	/cart	Add a product to cart
GET	/cart/<user_id>	View user's cart
PUT	/cart/<cart_id>	Update cart quantity
DELETE	/cart/<cart_id>	Remove cart item
POST	/orders	Place an order
GET	/orders/<user_id>	View user's orders

6. Database Design

The application uses four database models:

- User: Stores user name, email, and hashed password.
- Product: Stores product name, price, and available quantity.
- Cart: Stores user ID, product ID, and selected quantity.

- Order: Stores user ID, total amount, and order status.

The database is configured using SQLite and Flask-SQLAlchemy.

7. Installation and Execution

1. Install Python on the system.
2. Install the required packages:

pip install flask flask-sqlalchemy

3. Save the application code in E-Commerce Backend.py.
4. Run the application:

python "E-Commerce Backend.py"

5. Open Thunder Client in VS Code.
6. Send API requests to http://127.0.0.1:5000.

8. Testing

The APIs can be tested using Thunder Client by selecting the appropriate HTTP method, entering the endpoint URL, and providing JSON request data when required.

Test cases include:

- Successful user registration and login.
- Viewing and managing products.
- Adding and updating cart items.
- Checking cart totals.
- Placing orders and viewing order history.
- Handling invalid user, product, and quantity requests.

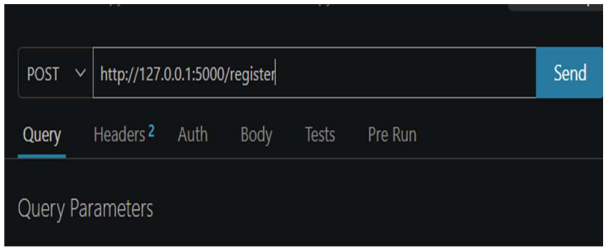
9. Limitations

- The project does not include a graphical frontend.
- Payment gateway integration is not implemented.
- Authentication and authorization are not fully implemented.

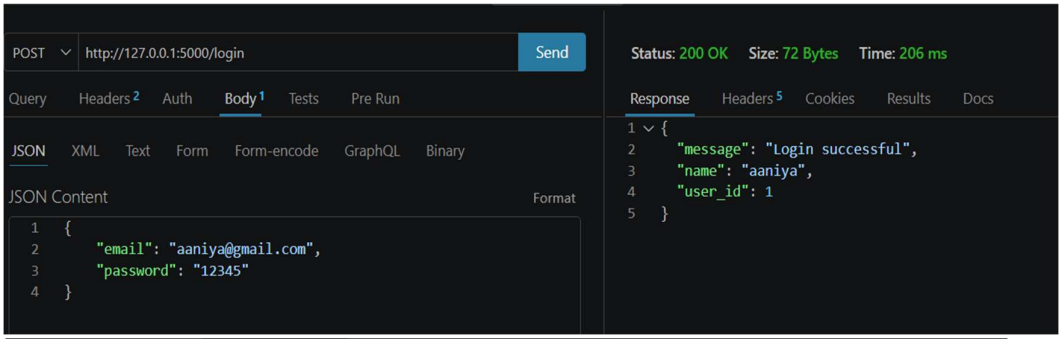
- The application is intended for learning and demonstration, not direct production deployment.

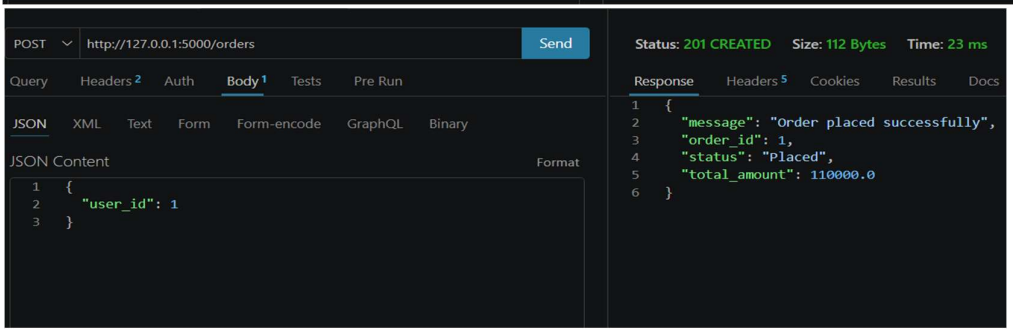
10. Screen Shot

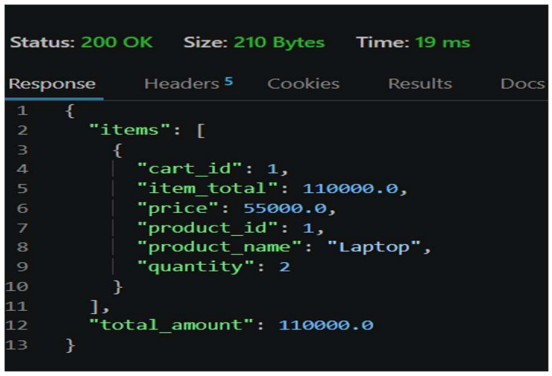












11. Conclusion

The E-Commerce Backend API demonstrates the use of Flask, SQLite, and REST API concepts to manage users, products, shopping carts, and orders. The project provides practical experience in backend development, database operations, and API testing.

12. GitHub Repository Link: -

https://github.com/saniya2698/E-Commerce-Backend-API